Python for a Quant Developer Interview

Core language, NumPy, the GIL, and the C++ boundary

- Scope
- Part 1 — Core Language
  - 1.1 Mutable vs immutable
  - 1.2 `is` vs `==`
  - 1.3 The data model
  - 1.4 Context managers
  - 1.5 Generators
  - 1.6 Decorators
  - 1.7 `dict.get` vs `[]` — and the C++ mirror image
  - 1.8 Common gotchas
- Part 2 — The GIL and Concurrency
  - 2.1 What the GIL is
  - 2.2 Releasing the GIL from C++
  - 2.3 The alternatives
- Part 3 — NumPy
  - 3.1 Vectorise
  - 3.2 Views versus copies
  - 3.3 Broadcasting
  - 3.4 Structured arrays — and the bug you hit
  - 3.5 dtypes and memory
- Part 4 — pandas
  - 4.1 The essentials
  - 4.2 Performance
  - 4.3 `SettingWithCopyWarning`
- Part 5 — The C++ Boundary
  - 5.1 The measurement that justifies everything
  - 5.2 pybind11 basics
  - 5.3 The alternatives
- Part 6 — Performance and Testing
  - 6.1 Profiling
  - 6.2 pytest
- Part 7 — Likely Questions

Scope

They named Python explicitly. For a quant dev role the bar is: **write clean idiomatic Python, know why it's slow and what to do about it, and understand the boundary with C++.** You are not being tested on Django.

Examples are drawn from your submission where possible, because a measured example beats a definition.

Part 1 — Core Language

1.1 Mutable vs immutable

Immutable	Mutable
`int`, `float`, `str`, `tuple`, `frozenset`, `bytes`	`list`, `dict`, `set`, `bytearray`, most objects

Why it matters — the classic bug:

```
def add_item(item, items=[]):      # DANGEROUS
    items.append(item)
    return items

add_item(1)   # [1]
add_item(2)   # [1, 2]  <- same list, default evaluated ONCE at definition
```

Fix:

```
def add_item(item, items=None):
    if items is None:
        items = []
```

Tuples are hashable, lists aren't — so tuples can be dict keys and set members.

Careful: a tuple containing a list is *not* hashable. Immutability is shallow.

1.2 `is` vs `==`

`==` compares value; `is` compares identity.

```
a = [1,2]; b = [1,2]
a == b     # True
a is b     # False

x = 256; y = 256
x is y     # True   -- small ints are cached
x = 257; y = 257
x is y     # False  -- CPython detail, never rely on it
```

Only use `is` for `None`, `True`, `False` — those are singletons.

1.3 The data model

```
class Position:
    __slots__ = ('qty', 'avg_price')     # no __dict__: less memory, faster attribute access

    def __init__(self, qty, avg_price):
        self.qty = qty
        self.avg_price = avg_price

    def __repr__(self):                   # for developers -- should be unambiguous
        return f"Position(qty={self.qty}, avg_price={self.avg_price})"

    def __eq__(self, other):
        if not isinstance(other, Position):
            return NotImplemented
        return (self.qty, self.avg_price) == (other.qty, other.avg_price)

    def __hash__(self):                   # define WITH __eq__ or the object becomes unhashable
        return hash((self.qty, self.avg_price))
```

Methods worth knowing: `__init__`, `__repr__` vs `__str__`, `__eq__`/`__hash__`, `__len__`, `__getitem__`, `__iter__`/`__next__`, `__enter__`/`__exit__`, `__call__`.

`__slots__` is worth mentioning for a quant role — it removes the per-instance `__dict__`, saving memory and speeding

attribute access. Matters when you hold millions of objects.

Defining `__eq__` sets `__hash__` to None. If you want the object hashable you must define both.

1.4 Context managers

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc_val, tb):
        self.elapsed = time.perf_counter() - self.start
        return False        # False -> exceptions propagate; True -> suppressed

with Timer() as t:
    do_work()
```

`__exit__` runs **on every exit path**, including exceptions. This is Python's RAII.

Simpler form:

```
from contextlib import contextmanager

@contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield
    finally:
        print(time.perf_counter() - start)
```

1.5 Generators

```
def read_fills(path):
    with open(path) as f:
        for line in f:
            yield parse(line)      # one at a time, O(1) memory
```

A generator computes lazily. Memory is O(1) rather than O(n), which matters for a large tick file.

Generator expression vs list comprehension:

```
sum(x*x for x in range(10**7))     # generator: no list built
sum([x*x for x in range(10**7)])   # list: 10M elements materialised first
```

`yield from` delegates to a sub-generator.

1.6 Decorators

A decorator is a function returning a function:

```python
import functools

def timed(func):
    @functools.wraps(func)                # preserves __name__ and __doc__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter()-start:.6f}s")
        return result
    return wrapper

@timed
def compute(): ...
```

Always use `functools.wraps` — without it the wrapped function loses its name and docstring, which breaks introspection and debugging.

Useful built-ins: `@property`, `@staticmethod`, `@classmethod`, `@functools.lru_cache`, `@dataclass`.

1.7 `dict.get` vs `[]` — and the C++ mirror image

```python
d[key]                # raises KeyError if absent
d.get(key)            # returns None
d.get(key, 0)         # returns a default
d.setdefault(key, 0)  # returns AND INSERTS
```

The connection to your submission: the Python prototype used `dict.get(sym, 0)`, which reads **without** inserting. The C++ equivalent trap is `operator[]`, which **does** insert.

They look equivalent and behave oppositely. In Python `[]` raises; in C++ `[]` silently creates. That was the primary bug in two of the five problems.

`collections.defaultdict` inserts on access by design — useful, but know that it does.

1.8 Common gotchas

Late binding in closures:

```python
fs = [lambda: i for i in range(3)]
[f() for f in fs]        # [2, 2, 2]  -- all see the final i

fs = [lambda i=i: i for i in range(3)]
[f() for f in fs]        # [0, 1, 2]  -- bound at definition
```

Shallow vs deep copy:

```python
import copy
b = a.copy()             # shallow: container copied, elements shared
b = copy.deepcopy(a)     # recursive
```

Floats are IEEE-754 doubles — same as C++:

```python
0.1 + 0.2 == 0.3         # False
(0.1 + 0.2).hex()        # '0x1.3333333333334p-2'
(0.3).hex()              # '0x1.3333333333333p-2'
```

That `.hex()` comparison is exactly what your parity tests use, because “close enough” is the failure mode they exist to

catch.

For exact decimal arithmetic use `decimal.Decimal`; for exact rationals, `fractions.Fraction` (which is what you used to produce the Problem 4 reference numbers).

Part 2 — The GIL and Concurrency

2.1 What the GIL is

One lock protecting the CPython interpreter. **Only one thread executes Python bytecode at a time.**

It exists because CPython's reference counting isn't thread-safe — without the GIL every refcount increment would need an atomic operation, which would slow down single-threaded code substantially.

Workload	Do threads help?
CPU-bound pure Python	**No** — the GIL serialises it
I/O-bound	**Yes** — the GIL is released during I/O
NumPy / C extensions	**Yes, if the extension releases it**

Note: Python 3.13 shipped an experimental free-threaded build (PEP 703). Worth knowing it exists; not yet the default.

2.2 Releasing the GIL from C++

```
{
    py::gil_scoped_release release;   // RAII: reacquired at the closing brace
    heavy_cpp_work();                 // must touch NO Python objects
}
```

Safe only because nothing inside touches a `py::` object. Touching one after release is a **crash**, not a slowdown.

That's why the list-of-tuples batch path *cannot* release it — every element is a Python object unpacked inside the loop. The arrays path converts to `std::vector` **before** the body runs, so the loop is pure C++.

How you'd prove it: run a spinning Python thread alongside the batch call and count its iterations. If the GIL is genuinely released, it makes progress.

2.3 The alternatives

Approach	For
`threading`	I/O-bound work
`multiprocessing`	CPU-bound — separate processes, separate GILs. Cost: IPC and pickling.
`asyncio`	many concurrent I/O operations, single thread, cooperative
`concurrent.futures`	uniform interface over threads or processes
C extension	the real answer for CPU-bound numeric work

Part 3 — NumPy

3.1 Vectorise

```
returns = today / prev - 1.0                      # C speed
returns = [t/p - 1 for t, p in zip(today, prev)]  # ~100x slower
```

The loop version pays interpreter overhead per element. The vectorised version runs one C loop.

Rule of thumb: if you're writing a Python `for` loop over a NumPy array, you're probably doing it wrong.

3.2 Views versus copies

Slicing gives a view (shares memory). **Fancy indexing gives a copy.**

```
a = np.arange(10)
b = a[2:5]          # VIEW  -- modifying b modifies a
b[0] = 99           # a is now [0,1,99,3,...]

c = a[[2,3,4]]      # COPY  -- independent
d = a[a > 5]        # COPY  (boolean mask)
```

Check with `b.base is a` or `np.shares_memory(a, b)`.

Why it matters: an accidental view means one calculation silently corrupts another's input. An accidental copy means unexpected memory and time.

3.3 Broadcasting

Shapes align **from the right**; dimensions of size 1 stretch.

```
a = np.ones((3, 4))
b = np.ones(4)          # treated as (1,4) -> broadcast to (3,4)
a + b                   # works

c = np.ones(3)          # (3,) does NOT align with (3,4) from the right
a + c                   # ERROR
a + c[:, np.newaxis]    # (3,1) broadcasts -> works
```

3.4 Structured arrays — and the bug you hit

```
dt = np.dtype([("instrument","S24"), ("side","u1"),
               ("quantity","i8"), ("price","f8")])
arr = np.array([...], dtype=dt)
```

Two traps, both of which bit during your submission:

1. A field view is STRIDED, not contiguous. `arr["instrument"]` has a stride equal to the **full record size**, not the field's itemsize. Walking it in C++ with `base + i * itemsize` reads interleaved bytes from the neighbouring fields.

It produced *plausible-looking garbage* — symbols that were the right length and wrong content — and surfaced as a `UnicodeDecodeError` three test cases downstream of the actual bug. Fixed by using `strides[0]`.

2. `'S'` fields are NUL-padded, not NUL-terminated. A symbol that exactly fills the field has **no terminator at all**, so you must bound the search by itemsize.

3.5 dtypes and memory

```
a.dtype        # float64 by default
a.astype(np.float32)     # half the memory, ~7 significant digits
a.nbytes                 # actual bytes used
```

`np.float32` is worth considering when memory bandwidth dominates. Rarely right for money.

Contiguity: `a.flags['C_CONTIGUOUS']`. A transposed array is not C-contiguous, so passing it to C++ may force a copy — which is what `py::array::forcecast` does silently.

Part 4 — pandas

4.1 The essentials

```
df.loc[row_label, col_label]     # label-based
df.iloc[row_pos, col_pos]        # position-based

df.groupby('symbol').agg({'qty': 'sum', 'price': 'mean'})
pd.merge(a, b, on='symbol', how='left')       # inner / left / right / outer
df.resample('1min').ohlc()                    # time series
df.pivot_table(index='date', columns='symbol', values='close')
```

4.2 Performance

Never use `iterrows` — it's row-by-row Python and constructs a Series per row.

```
df['ret'] = df['close'] / df['prev'] - 1        # vectorised: fast
df['ret'] = df.apply(lambda r: ..., axis=1)     # ~100x slower
```

`df.values` or `df.to_numpy()` gives the underlying array — drop to NumPy when pandas overhead dominates.

4.3 `SettingWithCopyWarning`

```
subset = df[df.qty > 0]     # may be a view or a copy -- pandas can't tell you which
subset['x'] = 1             # WARNING: the write may not reach df
```

Fix: `.loc` in one step, or `.copy()` explicitly if you meant a copy.

Same underlying issue as NumPy views — you don't know whether you're aliasing.

Part 5 — The C++ Boundary

5.1 The measurement that justifies everything

200,000 fills ingested from Python:

Path	ns/fill	Speedup
Pure Python	545	1.0×
C++, one call per fill	**493**	**1.1×**
C++, list of tuples	316	1.7×
C++, batch arrays, GIL released	**104**	**5.3×**

A per-fill binding gets 1.1× on a core that's 45× faster. The entire gain is handed back in interpreter dispatch and argument conversion.

The pattern: convert at the boundary once, then release the GIL and run the whole loop with no Python involvement.

5.2 pybind11 basics

```
PYBIND11_MODULE(mymodule, m) {              // name MUST match the .so filename
    py::class_<Position>(m, "Position")
        .def_readonly("qty", &Position::qty)
        .def_property_readonly("notional", &Position::notional);

    py::class_<Tracker>(m, "Tracker")
        .def(py::init<>())
        .def("on_fill", &onFill, py::arg("instrument"), py::arg("side"));
}
```

Two real bugs from building it:

1. Default arguments are converted EAGERLY, at registration time. A type used as a default must already be registered:

```
ImportError: arg(): could not convert default argument 'limits: Limits'
             ... (type not registered yet?)
```

It fails at **import**, not at call. Registration order is semantic in pybind11, not cosmetic.

2. Exception types matter. `py::value_error` becomes a Python `ValueError`; `std::invalid_argument` becomes a generic `RuntimeError`.

5.3 The alternatives

Tool	Notes
pybind11	header-only, modern C++, the default choice
Cython	Python-like syntax compiled to C; good for gradual optimisation
ctypes / cffi	no compilation needed, but C ABI only and slower per call
Numba	JIT for numeric Python, no C++ at all
nanobind	pybind11's leaner successor, faster compiles

Part 6 — Performance and Testing

6.1 Profiling

```
python -m cProfile -s cumtime script.py      # function-level
kernprof -l -v script.py                     # line_profiler: line-level
python -m memory_profiler script.py
```

`timeit` for micro-benchmarks — it disables the GC and runs many iterations:

```
import timeit
timeit.timeit('f()', globals=globals(), number=10000)
```

Same discipline as C++: take the minimum of repeated runs, not the mean. And check component timings add up to end-to-end.

6.2 pytest

```
import pytest

@pytest.mark.parametrize("seed", range(20))
def test_fuzz(seed):
    ...

def test_raises():
    with pytest.raises(ValueError):
        f(-1)

@pytest.fixture
def tracker():
    return PositionTracker()
```

`pytest.approx` **for floats** — but note where you *shouldn't* use it. Your parity tests compare `.hex()` exactly, because "close enough" is precisely the failure mode being tested.

Part 7 — Likely Questions

"Why doesn't threading speed up my CPU-bound Python?" The GIL — only one thread executes bytecode at a time. Use multiprocessing, or push the work into C where the GIL can be released.

"When is it safe to release the GIL?" When the code holds it releases touches no Python objects. Convert to C++ types first, then release. Touching a Python object without the GIL is a crash, not a slowdown.

"List vs tuple?" Mutable vs immutable. Tuples are hashable so they can be dict keys. Tuples are slightly smaller and faster to construct.

"What's a generator and when would you use one?" A lazy iterator — computes on demand, so memory is O(1). Use it for large files or streams where you don't need everything at once.

"Shallow vs deep copy?" Shallow copies the container and shares the elements; deep copies recursively. `copy.copy` vs `copy.deepcopy`.

"How would you speed up slow Python?" Profile first — `cProfile` then `line_profiler`. Then in order: vectorise with NumPy, use better algorithms and data structures, and only then move the hot part to C++ **with a batch interface**, because a per-call binding gives almost nothing.

"View or copy in NumPy?" Slicing gives a view; fancy indexing and boolean masks give copies. Check with `np.shares_memory`. Getting it wrong means either silent aliasing or unexpected memory.

"How do you compare floats in a test?" Depends on intent. Computed values: `pytest.approx` with a relative tolerance. Values that must be bit-identical — validating a reimplementation, or a round-trip — compare `.hex()` exactly.

"What's `__slots__`?" Removes the per-instance `__dict__`. Less memory and faster attribute access, at the cost of not being able to add attributes dynamically. Worth it when you hold millions of instances.

"Decorator?" A function that takes a function and returns a function, used to wrap behaviour. Always use `functools.wraps` so the wrapped function keeps its name and docstring.

"What's the mutable default argument bug?" Defaults are evaluated **once, at function definition**. So a mutable default is shared across all calls. Use `None` and construct inside.